



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н. Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н. Э. Баумана)

---

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

---

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

---

# РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

## *К КУРСОВОЙ РАБОТЕ*

*НА ТЕМУ:*

*«Компилятор языка В»*

Студент ИУ7-22М  
(Группа)

\_\_\_\_\_  
(Подпись, дата)

Агарков М. В.  
(И. О. Фамилия)

Руководитель курсовой работы

\_\_\_\_\_  
(Подпись, дата)

Ступников А. А.  
(И. О. Фамилия)

*2025 г.*

# СОДЕРЖАНИЕ

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ</b>                                                   | <b>4</b>  |
| <b>1 Аналитический раздел</b>                                     | <b>5</b>  |
| 1.1 Этапы компиляции . . . . .                                    | 5         |
| 1.1.1 Лексический анализ . . . . .                                | 5         |
| 1.1.2 Синтаксический анализ . . . . .                             | 6         |
| 1.1.3 Семантический анализ . . . . .                              | 6         |
| 1.1.4 Генерация кода . . . . .                                    | 6         |
| 1.2 Методы реализации лексического и синтаксического анализаторов | 7         |
| 1.3 Алгоритмы лексического и синтаксического анализа . . . . .    | 7         |
| 1.4 Генераторы анализаторов . . . . .                             | 8         |
| 1.5 LLVM . . . . .                                                | 9         |
| 1.6 Выбор языка разработки . . . . .                              | 11        |
| 1.7 Выбор инструментов разработки . . . . .                       | 12        |
| <b>2 Конструкторский раздел</b>                                   | <b>15</b> |
| 2.1 Структура компилятора . . . . .                               | 15        |
| 2.2 Язык программирования В . . . . .                             | 16        |
| 2.3 Метод обхода AST . . . . .                                    | 18        |
| 2.4 Генерация LLVM IR . . . . .                                   | 19        |
| <b>3 Технологический раздел</b>                                   | <b>21</b> |
| 3.1 Сгенерированные классы анализаторов . . . . .                 | 21        |
| 3.2 Обнаружение ошибок . . . . .                                  | 21        |
| 3.3 Пример компиляции программы . . . . .                         | 21        |
| <b>Список использованных источников</b>                           | <b>24</b> |
| <b>ПРИЛОЖЕНИЕ А</b>                                               | <b>25</b> |
| <b>ПРИЛОЖЕНИЕ В</b>                                               | <b>30</b> |
| <b>ПРИЛОЖЕНИЕ С</b>                                               | <b>36</b> |

# ВВЕДЕНИЕ

Компилятор — это программа, переводящая написанный на языке программирования текст в набор машинных кодов. Целью данного курсового проекта является разработка компилятора для языка В.

В ходе выполнения курсового проекта необходимо решить следующие основные задачи:

- провести анализ предметной области;
- разработать блок лексического и синтаксического анализа с явным построением дерева разбора для заданного исходного кода;
- разработать блок семантического анализа и генерации кода LLVM IR.

# 1 Аналитический раздел

В данном разделе приводится краткий обзор предметной области. Рассмотрены основные этапы компиляции, а также проанализированы методы их реализации.

## 1.1 Этапы компиляции

Компиляция состоит из следующих основных этапов [1]:

- лексический анализ;
- синтаксический анализ;
- семантический анализ;
- генерация кода.

### 1.1.1 Лексический анализ

Задачей лексического анализа является аналитический разбор входной цепочки символов, составляющих текст компилируемой программы, с целью получения на выходе последовательности токенов. Также данный процесс называют «токенизацией».

Лексический анализатор функционирует в соответствии с некоторыми правилами построения допустимых входных последовательностей. Данные правила могут быть определены, например, в виде детерминированного конечного автомата, регулярного выражения или праволинейной грамматики. С практической точки зрения выяснено, что наиболее удобным способом является формализация работы лексического анализатора с помощью грамматики.

Лексический анализ может быть представлен и как самостоятельная фаза трансляции, и как составная часть фазы синтаксического анализа. В первом случае лексический анализатор реализуется в виде отдельного модуля, который принимает последовательность символов, составляющих текст компилируемой программы, и выдаёт список обнаруженных лексем. Во втором случае лексический анализатор фактически является подпрограммой, вызываемой синтаксическим анализатором для получения очередной лексемы [2].

В процессе лексического анализа обнаруживаются лексические ошибки — простейшие ошибки компиляции, связанные с наличием в тексте программы недопустимых символов, некорректной записью идентификаторов, числовых констант.

На этапе лексического анализа обычно также выполняются такие действия, как удаление комментариев и обработка директив условной компиляции.

### **1.1.2 Синтаксический анализ**

Синтаксический анализатор использует поток токенов, полученный на этапе лексического анализа для создания древовидного промежуточного представления, которое описывает грамматическую структуру потока токенов. Типичным представлением является абстрактное синтаксическое дерево, в котором каждый внутренний узел представляет операцию, а дочерние узлы — аргументы этой операции [3].

Последующие фазы компиляции используют грамматическую структуру, которая помогает проанализировать исходную программу и сгенерировать целевую.

На этапе синтаксического анализа фиксируются синтаксические ошибки, то есть ошибки, связанные с нарушением принятой структуры программы.

### **1.1.3 Семантический анализ**

Семантический анализатор используя синтаксическое дерево, проверяет исходную программу на семантическую согласованность с определением языка. Он также собирает информацию о всех встреченных типах и сохраняет ее для последующего использования в процессе генерации промежуточного кода.

Таким образом, семантический анализатор предназначен для нахождения семантических ошибок и накопления данных о переменных, функциях и используемых типах, используемых при генерации кода.

### **1.1.4 Генерация кода**

Генерация кода — это последний этап процесса компиляции. Зачастую данный этап компиляции включает в себя и машинно-зависимую оптимизацию кода. На этом этапе компилятор генерирует машинно-зависимый код. Генератор кода должен иметь представление о среде выполнения целевой

машины и её наборе команд.

На этом этапе компилятор выполняет несколько основных задач:

- выбор инструкций — какую инструкцию использовать;
- создание расписания инструкций — в каком порядке должны быть упорядочены инструкции;
- распределение регистров — выделение переменных в регистры процессора;
- отладка данных — отладка кода с помощью отладочных данных.

Итоговый машинный код, сгенерированный генератором кода, может быть выполнен на целевой машине. Именно так высокоуровневый исходный код, преобразуется в формат, который можно запустить на любой целевой машине.

## **1.2 Методы реализации лексического и синтаксического анализаторов**

Реализовать лексический и синтаксический анализаторы можно двумя способами:

- используя стандартные алгоритмы анализа;
- с помощью инструментов генерации анализаторов.

## **1.3 Алгоритмы лексического и синтаксического анализа**

Существует два основных подхода к проведению синтаксического анализа: нисходящий анализ и восходящий.

Нисходящий синтаксический анализ решает задачу построения дерева разбора в прямом порядке обхода (обход в глубину). То есть реализует поиск левого порождения входной строки [1]. В общем виде нисходящий анализ представлен в анализе методом рекурсивного спуска, который может использовать откаты, т.е. производить повторный просмотр считанных символов.

Восходящий синтаксический анализ соответствует построению дерева разбора для входной строки, начиная с листьев (снизу) и идя по направлению к корню (вверх). Общий вид восходящего синтаксического анализа известен как анализ типа «перенос/свертка» [1]. На каждом шаге свертки некоторая подстрока, соответствующая правой части продукции, замещается левым символом данной продукции.

## 1.4 Генераторы анализаторов

Среди наиболее используемых генераторов анализаторов выделяются: Lex и Yacc, Accent, ANTLR.

Генератор синтаксических анализаторов YACC (Yet Another Compilers' Compiler) — это программа, которая строит LALR-анализаторы. На вход YACC получает описание грамматики в форме, близкой к форме Бэкуса-Наура и некоторую дополнительную информацию. Выходом является программа на языке Си, реализующая восходящий разбор. Как правило, Yacc применяется в сочетании с Lex — стандартным генератором лексических анализаторов.

Accent не полагается на определенные подклассы контекстно-свободных грамматик. Язык спецификаций Accent аналогичен языку Yacc, но Accent использует символические имена для атрибутов вместо чисел. Это позволяет записывать вашу грамматику в расширенной форме Бэкуса-Наура, в которой можно указывать повторения, варианты выбора и необязательные части, не вводя специальных правил для этих целей. Accent также можно использовать в сочетании с Lex.

ANTLR (ANother Tool for Language Recognition) — мощный генератор синтаксического анализатора для чтения, обработки, исполнения или перевода структурированных текстовых или двоичных файлов. Он широко используется для создания языков, инструментов и фреймворков.

Из грамматики ANTLR генерирует синтаксический анализатор, который может строить и обходить деревья синтаксического анализа. На основе правил заданной в виде РБНФ грамматики ANTLR генерирует классы нисходящего рекурсивного синтаксического анализатора. При этом обход дерева разбора можно выполнить с использованием паттернов посетитель или слушатель [4].

## 1.5 LLVM

LLVM [5] — проект программной инфраструктуры для создания компиляторов и сопутствующих им утилит. Состоит из набора компиляторов из языков высокого уровня (так называемых «фронтендов»), системы оптимизации, интерпретации и компиляции в машинный код.

В основе инфраструктуры используется RISC-подобная платформонезависимая система кодирования машинных инструкций (байткод LLVM IR), которая представляет собой высокоуровневый ассемблер.

Написан на C++, обеспечивает оптимизации на этапах компиляции, компоновки и исполнения. Изначально в проекте были реализованы компиляторы для языков Си и C++ при помощи фронтенда Clang, позже появились фронтенды для множества языков, в том числе: ActionScript, Ада, C#, Common Lisp, Crystal, CUDA, D, Delphi, Dylan, Fortran, Graphical G Programming Language, Halide, Haskell, Java (байткод), JavaScript, Julia, Kotlin, Lua, Objective-C, OpenGL Shading Language, Ruby, Rust, Scala, Swift, Xojo.

LLVM может создавать машинный код для множества архитектур, в том числе ARM, x86, x86-64, PowerPC, MIPS, SPARC, RISC-V и других (включая GPU от Nvidia и AMD).

Некоторые проекты имеют собственные LLVM-компиляторы (например LLVM-версия GCC), другие используют инфраструктуру LLVM, как например Glasgow Haskell Compiler.

Разработка начата в 2000 году в Университете Иллинойса. К середине 2010-х годов LLVM получил широкое распространение в индустрии: использовался, в том числе, в компаниях Adobe, Apple и Google. В частности, на LLVM основана подсистема OpenGL в Mac OS X 10.5, а iPhone SDK использует препроцессор (фронтенд) GCC с бэкэндом на LLVM. Apple и Google являются одними из основных спонсоров проекта, а один из основных разработчиков — Крис Латтнер — 11 лет проработал в Apple (с 2017 года — в Tesla Motors, с 2020 года — в разработчике процессоров и микроконтроллеров на архитектуре RISC-V SiFive).

LLVM поддерживает следующие типы данных:

- целые числа произвольной разрядности;
- числа с плавающей точкой;



- пустое значение `void`;
- массивы;
- структуры;
- вектора для упрощения SIMD-операций;
- функции.

Большинство инструкций в LLVM принимает два аргумента (операнда) и возвращает одно значение (трёхадресный код).

Значения определяются текстовым идентификатором. Локальные значения обозначаются префиксом `%`, а глобальные — `@`.

Тип операндов всегда указывается явно и однозначно определяет тип результата. Операнды арифметических инструкций должны иметь одинаковый тип, но сами инструкции «перегружены» для любых числовых типов и векторов.

LLVM поддерживает полный набор арифметических операций, побитовых логических операций и операций сдвига, а также специальные инструкции для работы с векторами.

LLVM IR строго типизирован, поэтому существуют операции приведения типов, которые явно кодируются специальными инструкциями. Набор из 9 инструкций покрывает все возможные приведения между различными числовыми типами: целыми и с плавающей точкой, со знаком и без, различной разрядности и пр. Кроме этого, есть инструкции преобразования между целыми и указателями, а также универсальная инструкция для приведения типов `bitcast` (ответственность за корректность таких преобразований возлагается на программиста).

Помимо значений-регистров, в LLVM есть и работа с памятью. Значения в памяти адресуются типизированными указателями. Обратиться к памяти можно с помощью двух инструкций: `load` и `store`.

Инструкция `alloca` выделяет память на стеке. Память, выделенная `alloca`, автоматически освобождается при выходе из функции при помощи инструкций `ret` или `unwind`.

Для вычисления адресов элементов массивов, структур и т. д. с правильной типизацией используется инструкция `getelementptr`. `getelementptr`

только вычисляет адрес, но не обращается к памяти. Инструкция принимает произвольное количество индексов и может разыменовывать структуры любой вложенности.

Также существует инструкции `extractvalue` и `insertvalue`. Они отличаются от `getelementptr` тем, что принимают не указатель на агрегатный тип данных (массив или структуру), а само значение такого типа. `extractvalue` возвращает соответствующее значение подэлемента, а `insertvalue` порождает новое значение агрегатного типа.

## 1.6 Выбор языка разработки

Разработка компилятора требует от выбранного языка программирования высокой производительности, контроля над памятью, возможности тонкой настройки структур данных, а также наличия развитой экосистемы и совместимости с существующими инструментами анализа и трансляции кода. По совокупности этих факторов язык C++ является одним из лучших вариантов для реализации компиляторных технологий.

1. **Высокая производительность C++** — это компилируемый язык низкого уровня по сравнению с большинством современных высокоуровневых языков. Он позволяет писать максимально эффективный по скорости выполнения код, что критически важно для компилятора, который должен обрабатывать исходные тексты программ, зачастую большие по объёму, с высокой скоростью и минимальными накладными расходами. Особенно это актуально при генерации промежуточного или машинного кода, где даже небольшие задержки масштабируются на сотни или тысячи файлов проекта.
2. **Контроль над памятью** Компилятор — это тип приложения, который активно оперирует абстрактными синтаксическими деревьями, таблицами символов, графами потока управления и промежуточным представлением (IR). Работа с такими структурами требует тонкого контроля над выделением и освобождением памяти. Явное управление памятью в C++ позволяет эффективно реализовывать аллокаторы, буферы, арены памяти и т. д., а также предотвращать утечки за счёт использования умных указателей `std::unique_ptr`, `std::shared_ptr`.

3. **Совместимость с LLVM и другими библиотеками C++** является основным языком, на котором написана платформа LLVM — одна из самых популярных и современных инфраструктур для разработки компиляторов. Использование C++ позволяет напрямую использовать API LLVM, не прибегая к оберткам, биндингам или дополнительным прослойкам. Это значительно упрощает реализацию фаз оптимизации, генерации машинного кода и работы с промежуточным представлением LLVM IR.
4. **Поддержка метапрограммирования** Современные версии C++ (начиная с C++11 и далее) предоставляют расширенные возможности метапрограммирования, которые позволяют реализовать механизмы генерации кода, оптимизации и шаблонов проектирования на этапе компиляции. Это особенно ценно при создании компиляторов, где производительность и модульность критичны.

## 1.7 Выбор инструментов разработки

ANTLR (ANother Tool for Language Recognition) был выбран в качестве основного инструмента для реализации лексического и синтаксического анализа в рамках разработки компилятора языка В. Этот выбор обусловлен совокупностью факторов, обеспечивающих удобство, гибкость и соответствие современным требованиям к построению языков программирования. В отличие от классических генераторов парсеров, таких как Lex, YACC, Bison или Accent, ANTLR предлагает более высокоуровневый и выразительный подход к описанию грамматик. Он использует стратегию LL(\*)-анализа, позволяющую описывать конструкции языка в форме, близкой к естественной, без необходимости сложных преобразований грамматик и устранения левой рекурсии. Благодаря этому процесс разработки становится более наглядным и менее подверженным синтаксическим ошибкам.

Классические инструменты, такие как Lex и YACC, требуют отдельного описания лексического и синтаксического анализаторов, что приводит к усложнению структуры проекта и затрудняет сопровождение. Они также ориентированы на LALR(1)-грамматики, что накладывает ограничения на допустимые конструкции языка и требует ручного вмешательства при добавлении новых правил. Кроме того, интеграция с современными средства-

ми разработки для таких инструментов ограничена, они не предоставляют средств визуализации дерева разбора или отладки грамматики. Bison, являющийся развитием YACC, сохраняет многие из этих ограничений и требует значительных усилий при сопровождении грамматик. Инструмент Accent, хотя и разрабатывался как более современная альтернатива, в настоящее время практически не поддерживается, имеет крайне ограниченное сообщество пользователей и минимальный набор документации, что делает его непригодным для учебных и исследовательских проектов.

ANTLR, напротив, предоставляет интегрированную среду для описания лексических и синтаксических правил, автоматически генерирует абстрактное синтаксическое дерево и предлагает стандартные шаблоны обхода дерева через Listener и Visitor API. Это позволяет чётко разделить синтаксическую структуру и семантическую обработку, способствуя модульности и масштабируемости кода. Кроме того, ANTLR поддерживает генерацию парсеров для нескольких языков программирования, включая C++, что делает его удобным для интеграции с системами на низком уровне и для реализации компиляторов с высокой производительностью. Он обладает развитой инфраструктурой, активным сообществом и интеграцией с популярными средами разработки, такими как IntelliJ IDEA и Visual Studio Code, что облегчает отладку грамматик и сопровождение проекта в целом.

Таким образом, выбор ANTLR в качестве инструмента синтаксического анализа оправдан его преимуществами в читаемости, расширяемости, интеграции с современными средствами разработки и поддержке актуальных подходов к проектированию языков. Учитывая ограничения, присущие альтернативным решениям, использование ANTLR является наиболее целесообразным выбором в условиях данной задачи.

Для генерации промежуточного представления (IR) был выбран LLVM, поскольку на сегодняшний день это наиболее развитая и широко поддерживаемая инфраструктура для компиляторов. LLVM предоставляет богатый набор инструментов и библиотек для анализа, оптимизации и генерации кода, а его IR обладает строгой формальной семантикой и легко адаптируется под различные целевые архитектуры. В отличие от менее универсальных решений, таких как GCC RTL или самописные IR-форматы, LLVM активно развивается, имеет мощную экосистему и широко используется в промышленности (напри-

мер, в компиляторах Clang, Swift, Rust и др.). Альтернативы, сопоставимые по функциональности, зрелости и уровню поддержки, в настоящее время фактически отсутствуют, что делает LLVM практически безальтернативным выбором при создании современного компилятора.

## 2 Конструкторский раздел

### 2.1 Структура компилятора

Концептуальная модель разрабатываемого компилятора в нотации IDEF0 представлена на рисунке.

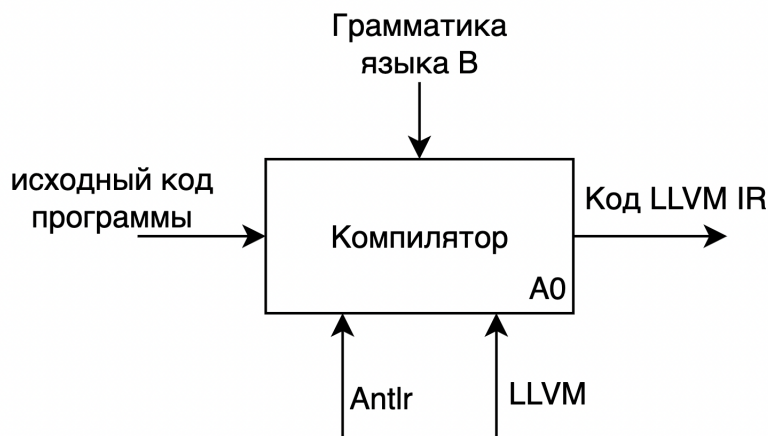


Рисунок 2.1 – Концептуальная модель в нотации IDEF0

На рисунке представлена детализированная концептуальная модель в нотации IDEF0. ANTLR фронтенд получает на вход исходный код программы и генерирует конкретное синтаксическое дерево, на основе которого компилятором генерируется LLVM IR.

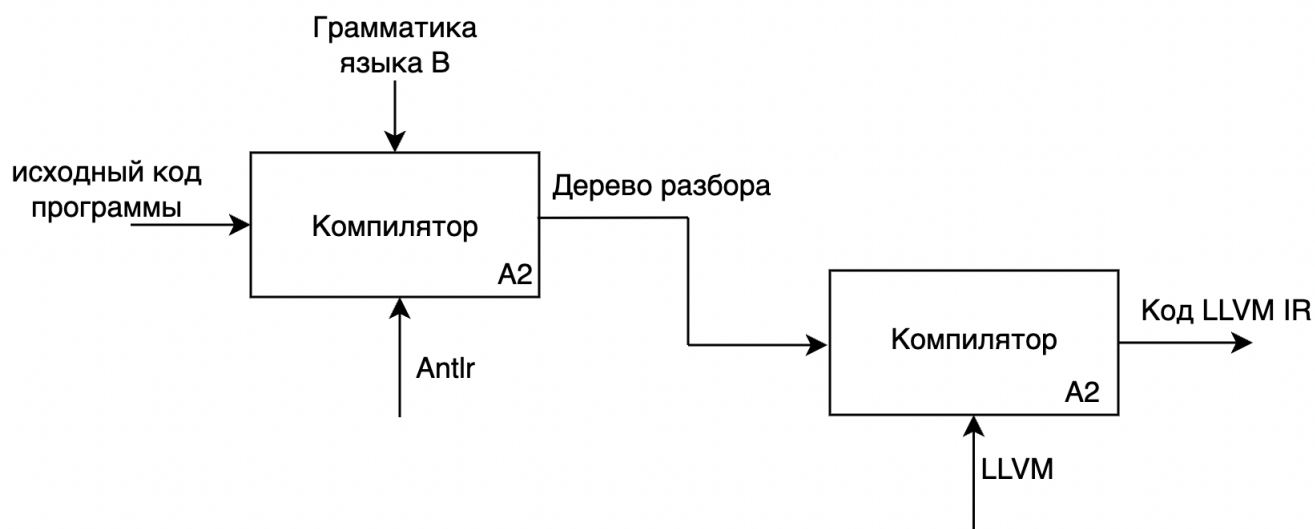


Рисунок 2.2 – Детализированная концептуальная модель в нотации IDEF0

## 2.2 Язык программирования В

Язык программирования В представляет собой упрощённый процедурный язык, исторически разработанный как предшественник языка С. В языке В отсутствует поддержка структур, модульности или объектно-ориентированных конструкций, однако активно используются управляющие конструкции (циклы, условные операторы, переходы), поэтому компилятору необходимо эффективно обрабатывать вложенные блоки, `jump`-операции (включая `goto`) и переменные области видимости. Основными особенностями языка являются:

### 1. Неявная типизация

Одной из ключевых особенностей языка программирования В, непосредственно влияющей на реализацию компилятора, является неявная типизация переменных при их объявлении. В соответствии с грамматикой языка, переменные могут быть объявлены с использованием ключевого слова `auto` без указания типа и без инициализации, например: `auto a;`. В этом случае тип переменной не фиксируется на момент объявления и остаётся неопределённым до момента первого присваивания, например: `a = 5;`. Тип переменной выводится на основе правой части выражения присваивания, и именно тогда происходит выделение памяти с использованием подходящего LLVM-типа. Для реализации такого поведения компилятор использует специальный механизм отслеживания неинициализированных переменных: все переменные, объявленные через `auto` без присваивания, помещаются во внутренний список. При встрече с операцией присваивания компилятор проверяет, содержится ли имя переменной в этом списке. Если переменная обнаружена, осуществляется анализ типа правой части выражения, после чего переменной выделяется память соответствующего типа (например, `i32` для целых значений) с помощью инструкции `alloca`, и она удаляется из списка неинициализированных. Если же переменная уже была проинициализирована ранее, выполняется обычное присваивание. Такой подход позволяет гибко и прозрачно реализовать ленивую типизацию переменных, сохраняя поведение языка и обеспечивая корректную генерацию промежуточного представления.

## 2. Отсутствие возвращаемого значения функции в ее сигнатуре

Ещё одной характерной особенностью языка В является отсутствие явно определяемого возвращаемого значения в сигнатуре функции. В отличие от большинства современных языков программирования, таких как С, С++ или Java, где возвращаемый тип указывается до имени функции, язык В вовсе не предусматривает подобной конструкции на синтаксическом уровне. Это решение значительно упрощает синтаксис и семантический анализ, избавляя компилятор от необходимости отслеживания типа результата и сопоставления его с выражением в операторе `return`. Однако такая упрощённость накладывает и определённые ограничения: функции в языке В могут использоваться лишь как подпрограммы, выполняющие побочные действия (например, модификация переменных, вывод на экран и т. д.), но не как выражения, результат которых может быть использован в более широком контексте. При реализации компилятора данная особенность позволяет отказаться от поддержки стека возвращаемых значений и существенно упростить логику генерации промежуточного представления, так как отсутствует необходимость управлять возвратом значения из функции через регистры или выделенную память. С точки зрения концептуального дизайна, это сближает язык В с процедурными языками раннего поколения и подчёркивает его ориентированность на обучение принципам трансляции, а не на построение полнофункционального языка общего назначения.

## 3. Поддержка только двух типов переменных: INT и STRING

Язык программирования В характеризуется минималистичной системой типов: он поддерживает лишь два основных типа переменных — целочисленный (INT) и строковый (STRING). Такое упрощение типовой системы было заложено в грамматике языка и отражает его учебно-исследовательскую направленность. Несмотря на кажущуюся ограниченность, это решение обладает важными импликациями для реализации компилятора. Во-первых, упрощается процесс синтаксического и семантического анализа: отсутствует необходимость в реализации сложных правил приведения типов, проверок на совместимость и перегрузок операций. Во-вторых, достаточно однозначно определяется



поведение операций: арифметические и логические действия применимы только к значениям типа `INT`, тогда как строковые значения допускаются, например, только в контексте присваивания или вывода. Тем не менее, ограниченный типовой ландшафт требует особого внимания при генерации промежуточного представления: компилятору необходимо чётко различать указатели на строки (часто реализуемые через `CreateGlobalString` в LLVM) и аллоцированные значения `i32`, обеспечивая при этом корректное размещение, обращение и копирование данных. Такая строгость в типизации при минимальном числе доступных типов служит хорошей основой для освоения ключевых принципов компиляции без усложнения логики трансляции.

Полная грамматика языка В представлена в приложении А.

## 2.3 Метод обхода AST

Для обхода абстрактного синтаксического дерева, построенного ANTLR, использовался паттерн посетитель (Visitor). ANTLR генерирует базовый класс `BaseVisitor`, в котором для каждого правила грамматики определён метод `visit<RuleName>`. Эти методы вызываются вручную при рекурсивном обходе дерева и могут быть переопределены для обработки конкретных узлов.

В отличие от слушателя, посетитель реализует явный контроль обхода: вызовы `visit` выполняются программно, что даёт большую гибкость при интерпретации или трансформации дерева. Это позволяет, например, изменять порядок обхода или реализовывать короткую обработку без посещения поддеревьев.

ANTLR генерирует интерфейс слушателя специфичный для заданной грамматики и содержащий `exit`- и `enter`-методы для каждого правила, которые нужно реализовать в зависимости от решаемой задачи. На рисунках ?? и ?? представлен порядок обхода дерева средствами ANTLR и порядок вызовов коллбэк-методов класса слушателя.



Рисунок 2.3 – Порядок обхода дерева разбора

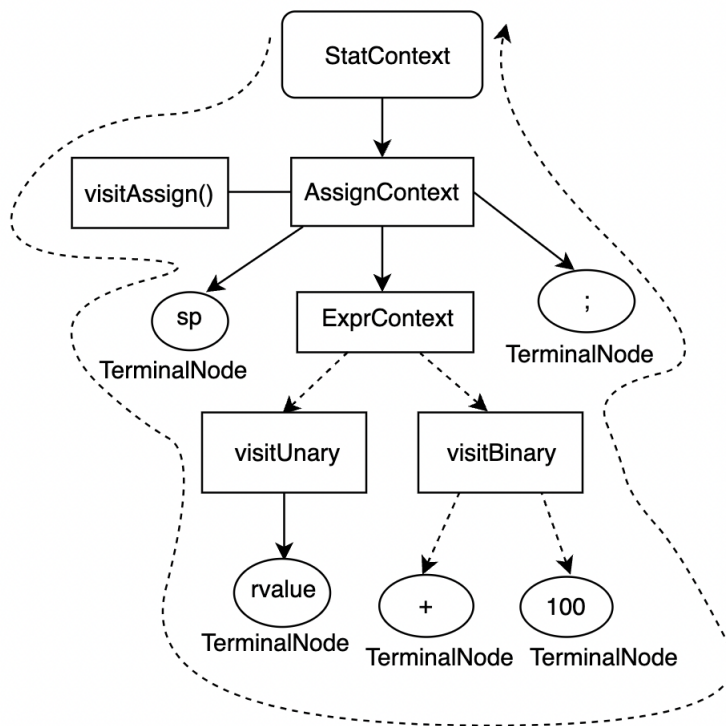


Рисунок 2.4 – Порядок вызова методов визитера

## 2.4 Генерация LLVM IR

Для генерации кода LLVM IR необходимо произвести обход дерева разбора, полученного из парсера ANTLR, и в каждой вершине произвести генерацию соответствующую её типу.

Схема алгоритма для генерации кода LLVM IR для цикла `for` с выражениями, выполняющимися до входа в цикл и после выполнения тела цикла, приведена на рисунках 4 и 5.

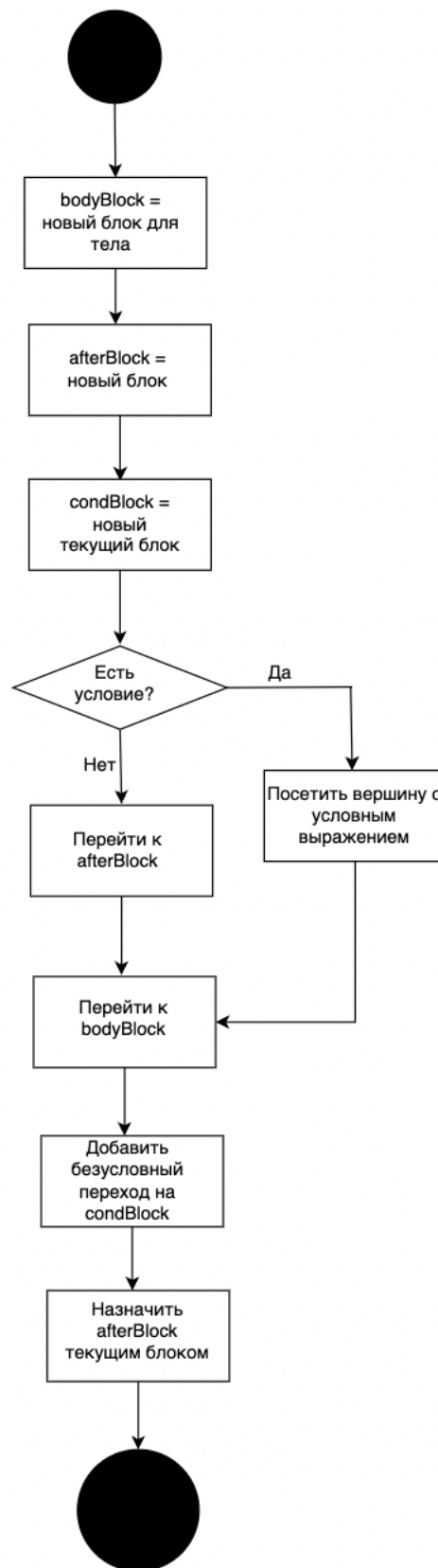


Рисунок 2.5 – Схема генерации кода для тела и выражения, выполняющегося после тела цикла while

## 3 Технологический раздел

### 3.1 Сгенерированные классы анализаторов

В результате работы ANTLR будут сгенерированы следующие классы:

- `bLexer` — лексический анализатор.
- `bParser` — синтаксический анализатор.
- `bVisitor` — абстрактный класс посетителя для обхода дерева.
- `bListener` — абстрактный класс слушателя, с пустыми методами.
- `bBaseVisitor` - реализация интерфейса `bVisitor` с пустыми переопределенными методами.
- `bBaseListener` - реализация интерфейса `bListener` с пустыми переопределенными методами.

Для обхода дерева используется паттерн `Visitor`: создаётся класс-наследник от `bVisitor`, в котором переопределяются методы `visit<RuleName>`. Каждый из этих методов вызывается вручную при обходе дерева, а возврат значений и переход к дочерним узлам осуществляется через вызов `visit(ctx>child)` или `visitChildren(ctx)`. Это позволяет точно управлять порядком обхода и обработкой содержимого узлов дерева, соответствующих правилам исходной грамматики.

### 3.2 Обнаружение ошибок

Все ошибки, обнаруженные на этапах лексического и синтаксического анализов, обрабатываются средствами ANTLR. Возникающие при этом исключения обрабатываются и выводится текст соответствующей ошибки. Также при построении графа вызовов обрабатываются ошибки вызова функции до ее объявления.

### 3.3 Пример компиляции программы

Пример программы, написанный на рассматриваемом языке В представлен на листинге 3.14. Данная программа реализует сортировку пузырьком.

Листинг 3.1 – Программа на рассматриваемом языке В

```
main() {  
    auto i;  
    i = 0;  
    while (i <= 10) {  
        print(i);  
        i++;  
        i++;  
    }  
  
    auto j;  
    j = 0;  
    while (j <= 10) {  
        print(j);  
        j++;  
    }  
  
    auto k;  
    k = 0;  
}
```

В листинге в приложении В приведен сгенерированный LLVM IR код для представленной на листинге программы.

## ЗАКЛЮЧЕНИЕ

В ходе работы над данным проектом был проведён анализ предметной области, разработан блок лексического и синтаксического анализа с явным построением дерева разбора для заданного исходного кода, разработан блок семантического анализа и генерации кода LLVM IR.

В результате чего был реализован компилятор языка В с использованием ANTLR и LLVM.

## Список литературы

- [1] Ахо А., Сети Р., Ульман Д. Компиляторы: принципы, технологии и инструментарий. – М.: ООО «И.Д. Вильямс», 2008. – 1184 с.
- [2] Серебряков В. А., Галочкин М. П. Основы конструирования компиляторов.
- [3] Ахо А., Ульман Дж. Теория синтаксического анализа, перевода и компиляции. Том 1. – М.: «Мир», 1978.
- [4] Parr T. The Definitive ANTLR4 Reference. – М.: Pragmatic Bookshelf, 2013.
- [5] Chris Lattner. The Design of LLVM // Dr. Dobb's Journal. – 2012.

## ПРИЛОЖЕНИЕ А

Листинг 3.2 – Грамматика ANTLR для языка В

```
grammar b;

program
    : definition* EOF
    ;

definition
    : name constant? (ival (',' ival)*)* ';'
    | name '(' (name (',' name)*)? ')' statement
    ;

ival
    : constant
    | name
    ;

statement
    : externsmt
    | autosmt
    | autoarraysmt
    | name ':' statement
    | casestmt
    | blockstmt
    | ifstmt
    | whilestmt
    | switchstmt
    | gotostmt
    | returnstmt
    | expressionstmt
    | nullstmt
    ;

nullstmt
    : ';'
    ;

expressionstmt
    : rvalue ';'
    ;
```



```

;

blockstmt
: '{' statement* '}'
;

returnstmt
: 'return' ((' rvalue '))? ';'
;

gotostmt
: 'goto' rvalue ';'
;

switchstmt
: 'switch' rvalue statement
;

whilestmt
: 'while' '(' rvalue ')' statement
;

ifstmt
: 'if' '(' rvalue ')' statement ('else' statement)?
;

casestmt
: 'case' constant ':' statement
;

externsmt
: 'extrn' name (',' name)* ';'
;

autosmt
: 'auto' name constant? (',' name constant?)* ';'
;

autoarraysmt
: 'auto' name '[' INT ']' ';'
;

```

```

rvalue
    : expression
    | comparison
    | ternary
    | assignment
    ;

ternary
    : expression '?' rvalue ':' rvalue
    ;

comparison
    : expression binary rvalue
    ;

assignment
    : name assign rvalue
    | name '[' rvalue ']' assign rvalue
    ;

expression
    : '(' rvalue ')'
    | name
    | constant
    | incdec name
    | name incdec
    | unary rvalue
    | '&' name
    | functioninvocation
    | name '[' rvalue ']'
    ;

functioninvocation
    : name '(' functionparameters? ')'
    ;

functionparameters
    : rvalue (',' rvalue)*
    ;

```

```

assign
    : '=' binary?
    ;

incdec
    : '++'
    | '--'
    ;

unary
    : '-'
    | '!'
    ;

binary
    : '|'
    | '&'
    | '=='
    | '!='
    | '<'
    | '<='
    | '>'
    | '>='
    | '<<'
    | '>>'
    | '-'
    | '+'
    | '%'
    | '*'
    | '/'
    ;

lvalue
    : name
    | '*' rvalue
    | name '[' INT ']'
    | rvalue '[' rvalue ']'
    ;

constant
    : INT

```

```

        | STRING1
        | STRING2
        ;

name
    : NAME
    ;

NAME
    : [a-zA-Z] [a-zA-Z0-9_]*
    ;

INT
    : [0-9]+
    ;

STRING1
    : '"' ~ ["\r\n"]* '"'
    ;

STRING2
    : '\'' ~ ['\r\n']* '\''
    ;

BLOCKCOMMENT
    : '/*' .*? '*/' -> skip
    ;

WS
    : [ \t\r\n] -> skip
    ;

```

## ПРИЛОЖЕНИЕ В

Листинг 3.3 – Тестовая программа №1

```
main() {  
    auto a;  
    a = 9;  
    auto b;  
    b = 34;  
    auto c;  
    c = 'abc';  
    print(a);  
    print(b);  
    print(c);  
    print(2);  
    print('abcd');  
    print(a);  
    print(b);  
}
```

Листинг 3.4 – Тестовая программа №2

```
main() {  
    auto a;  
    auto b;  
    a = 6;  
    b = 1981;  
  
    auto tmp;  
    tmp = max(a, b);  
    print(max(a, b));  
}
```

```
max(a, b) {  
    if (a > 0) {  
        print(1);  
    }  
    a = 3;  
    auto result;  
    result = a;  
    print(result);  
    print(a);  
    return (result);  
}
```

```
}
```

Листинг 3.5 – Тестовая программа №3

```
main() {  
    auto i;  
    i = 0;  
    while (i <= 10) {  
        print(i);  
        i++;  
        i++;  
    }  
  
    auto j;  
    j = 0;  
    while (j <= 10) {  
        print(j);  
        j++;  
    }  
  
    auto k;  
    k = 0;  
}
```

Листинг 3.6 – Тестовая программа №4

```
main() {  
    auto x;  
    x = 1;  
    ++x;  
    x++;  
    --x;  
    print(x);  
}
```

Листинг 3.7 – Тестовая программа №5

```
main() {  
    auto n;  
    n = 5;  
    while (n != 0) {  
        print(n);  
        n--;  
    }  
    f(24, 42);  
}
```

```

        return (0);
    }

    f(s, y) {
        print(s);
        print(y);
        return (0);
    }

```

Листинг 3.8 – Тестовая программа №6

```

main() {
    Print1();
    Print2(31415);
    Print3(777, 34);
}

Print1() {
    print("Hi!");
}

Print2(x) {
    print(x);
}

Print3(x, y) {
    print(x);
    print(y);
}

```

Листинг 3.9 – Тестовая программа №7

```

max(a, b) {
    auto result;
    result = a > b ? a : b;
    print(a);
    print(b);
    print(result);
}

main() {
    max(10, 20);
}

```

Листинг 3.10 – Тестовая программа №8

```
main() {
    someFunc(1982, 3);
    someFunc(0, 0);
}

someFunc(a, b) {
    if ((a + b) > 0) {
        print("a + b > 0");
    }
    else {
        print("a + b <= 0");
    }
}
```

Листинг 3.11 – Тестовая программа №9

```
main() {
    auto x;
    x = 2;

    switch x {
        case 1: {
            print(100);
        }
        case 2: {
            print(200);
        }
        case 3: {
            print(300);
        }
    }
}
```

Листинг 3.12 – Тестовая программа №10

```
main() {
    auto x;
    x = 5;
    goT0func(0);
    x = -3;
    goT0func(4);
}
```



```

goTOfunc(y) {
    if (y > 0) {
        goto label1;
    }
    print(0);
    label1 : print(42);
}

```

Листинг 3.13 – Тестовая программа №11

```

main() {
    auto x;
    auto y;
    auto max;
    max = 2;
    x = 10;
    y = 20;
    max = x > y ? x : y;
    print(max);
}

```

Листинг 3.14 – Тестовая программа №12

```

main()
{
    auto arr[5];
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 3;
    arr[3] = 4;
    arr[4] = 5;

    auto i;
    auto j;
    auto tmp1;
    auto tmp2;

    i = 0;
    j = 4;

    auto tmpCounter;
    tmpCounter = 0;

    while (tmpCounter < 5)

```

```

    {
        print(arr[tmpCounter]);
        ++tmpCounter;
    }

    while (i < j)
    {
        tmp1 = arr[i];
        tmp2 = arr[j];
        arr[i] = tmp2;
        arr[j] = tmp1;
        ++i;
        --j;
    }

    print(" ");

    tmpCounter = 0;
    while (tmpCounter < 5)
    {
        print(arr[tmpCounter]);
        ++tmpCounter;
    }
}

```

## ПРИЛОЖЕНИЕ С

Листинг 3.15 – Сгенерированный LLVM IR код для тестовой программы №1

```
; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"abc\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@3 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@4 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@5 = private unnamed_addr constant [5 x i8] c"abcd\00", align 1
@6 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@7 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@8 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %a = alloca i32, align 4
    store i32 9, ptr %a, align 4
    %loaded_from_var = load i32, ptr %a, align 4
    store i32 %loaded_from_var, ptr %a, align 4
    %b = alloca i32, align 4
    store i32 34, ptr %b, align 4
    %loaded_from_var1 = load i32, ptr %b, align 4
    store i32 %loaded_from_var1, ptr %b, align 4
    %loaded_int = load i32, ptr %a, align 4
    call void (ptr, ...) @printf(ptr @1, i32 %loaded_int)
    %loaded_int2 = load i32, ptr %b, align 4
    call void (ptr, ...) @printf(ptr @2, i32 %loaded_int2)
    call void (ptr, ...) @printf(ptr @3, ptr @0)
    call void (ptr, ...) @printf(ptr @4, i32 2)
    call void (ptr, ...) @printf(ptr @6, ptr @5)
    %loaded_int3 = load i32, ptr %a, align 4
    call void (ptr, ...) @printf(ptr @7, i32 %loaded_int3)
    %loaded_int4 = load i32, ptr %b, align 4
    call void (ptr, ...) @printf(ptr @8, i32 %loaded_int4)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

declare void @printf(ptr, ...)
```

## Листинг 3.16 – Сгенерированный LLVM IR код для тестовой программы №2

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@3 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@4 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@5 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %a = alloca i32, align 4
    store i32 6, ptr %a, align 4
    %loaded_from_var = load i32, ptr %a, align 4
    store i32 %loaded_from_var, ptr %a, align 4
    %b = alloca i32, align 4
    store i32 1981, ptr %b, align 4
    %loaded_from_var1 = load i32, ptr %b, align 4
    store i32 %loaded_from_var1, ptr %b, align 4
    %calltmp = call i32 @max(ptr %a, ptr %b)
    %tmp = alloca i32, align 4
    store i32 %calltmp, ptr %tmp, align 4
    %calltmp2 = call i32 @max(ptr %a, ptr %b)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @max(i32 %a, i32 %b) {
entry:
    %a1 = alloca i32, align 4
    store i32 %a, ptr %a1, align 4
    %b2 = alloca i32, align 4
    store i32 %b, ptr %b2, align 4
    %retval = alloca i32, align 4
    %tmp = alloca i32, align 4
    store i32 0, ptr %tmp, align 4
    %loaded_rhs = load i32, ptr %tmp, align 4
    %loaded_lhs = load i32, ptr %a1, align 4
    %gttmp = icmp sgt i32 %loaded_lhs, %loaded_rhs
    %ifcond = icmp ne i1 %gttmp, false
    br i1 %ifcond, label %if.then, label %if.end

```

```

if.then:                                     ; preds = %entry
    %tmp3 = alloca i32, align 4
    store i32 1, ptr %tmp3, align 4
    %loaded_int = load i32, ptr %tmp3, align 4
    call void (ptr, ...) @printf(ptr @0, i32 %loaded_int)
    br label %if.end

if.end:                                     ; preds = %if.then, %entry
    %a4 = alloca i32, align 4
    store i32 3, ptr %a4, align 4
    %loaded_from_var = load i32, ptr %a4, align 4
    store i32 %loaded_from_var, ptr %a4, align 4
    %result = alloca i32, align 4
    %loaded_rhs5 = load i32, ptr %a4, align 4
    store i32 %loaded_rhs5, ptr %result, align 4
    %loaded_int6 = load i32, ptr %result, align 4
    call void (ptr, ...) @printf(ptr @1, i32 %loaded_int6)
    %loaded_int7 = load i32, ptr %a4, align 4
    call void (ptr, ...) @printf(ptr @2, i32 %loaded_int7)
    %ret_val = load i32, ptr %result, align 4
    ret i32 %ret_val

entry8:                                     ; No predecessors!
    %a9 = alloca i32, align 4
    store i32 %a, ptr %a9, align 4
    %b10 = alloca i32, align 4
    store i32 %b, ptr %b10, align 4
    %retval11 = alloca i32, align 4
    %loaded_lhs12 = load i32, ptr %a9, align 4
    %gttmp13 = icmp sgt i32 %loaded_lhs12, 0
    %ifcond14 = icmp ne i1 %gttmp13, false
    br i1 %ifcond14, label %if.then15, label %if.end16

if.then15:                                 ; preds = %entry8
    call void (ptr, ...) @printf(ptr @3, i32 1)
    br label %if.end16

if.end16:                                  ; preds = %if.then15, %entry8
    %a17 = alloca i32, align 4
    store i32 3, ptr %a17, align 4
    %loaded_from_var18 = load i32, ptr %a17, align 4
    store i32 %loaded_from_var18, ptr %a17, align 4
    %result19 = alloca i32, align 4
    %loaded_rhs20 = load i32, ptr %a17, align 4
    store i32 %loaded_rhs20, ptr %result19, align 4
    %loaded_int21 = load i32, ptr %result19, align 4
    call void (ptr, ...) @printf(ptr @4, i32 %loaded_int21)

```

```

%loaded_int22 = load i32, ptr %a17, align 4
call void (ptr, ...) @printf(ptr @5, i32 %loaded_int22)
%ret_val23 = load i32, ptr %result19, align 4
ret i32 %ret_val23
}

```

```
declare void @printf(ptr, ...)
```

Листинг 3.17 – Сгенерированный LLVM IR код для тестовой программы №3

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %i = alloca i32, align 4
    store i32 0, ptr %i, align 4
    %loaded_from_var = load i32, ptr %i, align 4
    store i32 %loaded_from_var, ptr %i, align 4
    br label %while.cond

while.cond:                                     ; preds = %while.body, %entry
    %loaded_lhs = load i32, ptr %i, align 4
    %letmp = icmp sle i32 %loaded_lhs, 10
    %whilecond = icmp ne i1 %letmp, false
    br i1 %whilecond, label %while.body, label %while.end

while.body:                                     ; preds = %while.cond
    %loaded_int = load i32, ptr %i, align 4
    call void (ptr, ...) @printf(ptr @0, i32 %loaded_int)
    %loadtmp = load i32, ptr %i, align 4
    %inctmp = add i32 %loadtmp, 1
    store i32 %inctmp, ptr %i, align 4
    %loadtmp1 = load i32, ptr %i, align 4
    %inctmp2 = add i32 %loadtmp1, 1
    store i32 %inctmp2, ptr %i, align 4
    br label %while.cond

while.end:                                     ; preds = %while.cond
    %j = alloca i32, align 4
    store i32 0, ptr %j, align 4

```

```

%loaded_from_var3 = load i32, ptr %j, align 4
store i32 %loaded_from_var3, ptr %j, align 4
br label %while.cond4

while.cond4:                                     ; preds = %while.body5, %while.end
%loaded_lhs7 = load i32, ptr %j, align 4
%letmp8 = icmp sle i32 %loaded_lhs7, 10
%whilecond9 = icmp ne i1 %letmp8, false
br i1 %whilecond9, label %while.body5, label %while.end6

while.body5:                                     ; preds = %while.cond4
%loaded_int10 = load i32, ptr %j, align 4
call void @printf(ptr @1, i32 %loaded_int10)
%loadtmp11 = load i32, ptr %j, align 4
%inctmp12 = add i32 %loadtmp11, 1
store i32 %inctmp12, ptr %j, align 4
br label %while.cond4

while.end6:                                     ; preds = %while.cond4
%k = alloca i32, align 4
store i32 0, ptr %k, align 4
%loaded_from_var13 = load i32, ptr %k, align 4
store i32 %loaded_from_var13, ptr %k, align 4
%ret = load i32, ptr %retval, align 4
ret i32 %ret
}

declare void @printf(ptr, ...)

```

Листинг 3.18 – Сгенерированный LLVM IR код для тестовой программы №4

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
  "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
  %retval = alloca i32, align 4
  %x = alloca i32, align 4
  store i32 1, ptr %x, align 4
  %loaded_from_var = load i32, ptr %x, align 4
  store i32 %loaded_from_var, ptr %x, align 4
  %loadtmp = load i32, ptr %x, align 4
  %inctmp = add i32 %loadtmp, 1

```

```

    store i32 %inctmp, ptr %x, align 4
    %loadtmp1 = load i32, ptr %x, align 4
    %inctmp2 = add i32 %loadtmp1, 1
    store i32 %inctmp2, ptr %x, align 4
    %loadtmp3 = load i32, ptr %x, align 4
    %dectmp = sub i32 %loadtmp3, 1
    store i32 %dectmp, ptr %x, align 4
    %loaded_int = load i32, ptr %x, align 4
    call void (ptr, ...) @printf(ptr @0, i32 %loaded_int)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

```

```
declare void @printf(ptr, ...)
```

Листинг 3.19 – Сгенерированный LLVM IR код для тестовой программы №5

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %n = alloca i32, align 4
    store i32 5, ptr %n, align 4
    %loaded_from_var = load i32, ptr %n, align 4
    store i32 %loaded_from_var, ptr %n, align 4
    br label %while.cond

while.cond:                                     ; preds = %while.body, %entry
    %loaded_lhs = load i32, ptr %n, align 4
    %netmp = icmp ne i32 %loaded_lhs, 0
    %whilecond = icmp ne i1 %netmp, false
    br i1 %whilecond, label %while.body, label %while.end

while.body:                                     ; preds = %while.cond
    %loaded_int = load i32, ptr %n, align 4
    call void (ptr, ...) @printf(ptr @0, i32 %loaded_int)
    %loadtmp = load i32, ptr %n, align 4
    %dectmp = sub i32 %loadtmp, 1
    store i32 %dectmp, ptr %n, align 4

```



```

    br label %while.cond

while.end:                                     ; preds = %while.cond
    %calltmp = call i32 @f(i32 24, i32 42)
    ret i32 0
}

define i32 @f(i32 %s, i32 %y) {
entry:
    %s1 = alloca i32, align 4
    store i32 %s, ptr %s1, align 4
    %y2 = alloca i32, align 4
    store i32 %y, ptr %y2, align 4
    %retval = alloca i32, align 4
    %loaded_int = load i32, ptr %s1, align 4
    call void (ptr, ...) @printf(ptr @1, i32 %loaded_int)
    %loaded_int3 = load i32, ptr %y2, align 4
    call void (ptr, ...) @printf(ptr @2, i32 %loaded_int3)
    ret i32 0
}

declare void @printf(ptr, ...)

```

Листинг 3.20 – Сгенерированный LLVM IR код для тестовой программы №6

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"Hi!\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@3 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@4 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %calltmp = call i32 @Print1()
    %calltmp1 = call i32 @Print2(i32 31415)
    %calltmp2 = call i32 @Print3(i32 777, i32 34)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @Print1() {

```

```

entry:
    %retval = alloca i32, align 4
    call void (ptr, ...) @printf(ptr @1, ptr @0)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @Print2(i32 %x) {
entry:
    %x1 = alloca i32, align 4
    store i32 %x, ptr %x1, align 4
    %retval = alloca i32, align 4
    %loaded_int = load i32, ptr %x1, align 4
    call void (ptr, ...) @printf(ptr @2, i32 %loaded_int)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @Print3(i32 %x, i32 %y) {
entry:
    %x1 = alloca i32, align 4
    store i32 %x, ptr %x1, align 4
    %y2 = alloca i32, align 4
    store i32 %y, ptr %y2, align 4
    %retval = alloca i32, align 4
    %loaded_int = load i32, ptr %x1, align 4
    call void (ptr, ...) @printf(ptr @3, i32 %loaded_int)
    %loaded_int3 = load i32, ptr %y2, align 4
    call void (ptr, ...) @printf(ptr @4, i32 %loaded_int3)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

declare void @printf(ptr, ...)

```

Листинг 3.21 – Сгенерированный LLVM IR код для тестовой программы №7

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @max(i32 %a, i32 %b) {

```

```

entry:
    %a1 = alloca i32, align 4
    store i32 %a, ptr %a1, align 4
    %b2 = alloca i32, align 4
    store i32 %b, ptr %b2, align 4
    %retval = alloca i32, align 4
    %loaded_cond = load i32, ptr %b2, align 4
    %cond_bool = icmp ne i32 %loaded_cond, 0
    br i1 %cond_bool, label %ternary.true, label %ternary.false

ternary.true:                                ; preds = %entry
    br label %ternary.merge

ternary.false:                              ; preds = %entry
    br label %ternary.merge

ternary.merge:                              ; preds = %ternary.false, %ternary.true
    %ternarytmp = phi ptr [ %a1, %ternary.true ], [ %b2, %ternary.false ]
    %loaded_lhs = load i32, ptr %a1, align 4
    %loaded_rhs = load i32, ptr %ternarytmp, align 4
    %gttmp = icmp sgt i32 %loaded_lhs, %loaded_rhs
    %select_result = select i1 %gttmp, ptr %a1, ptr %b2
    %selected_loaded = load i32, ptr %select_result, align 4
    %result = alloca i32, align 4
    store i32 %selected_loaded, ptr %result, align 4
    %loaded_int = load i32, ptr %a1, align 4
    call void (ptr, ...) @printf(ptr @0, i32 %loaded_int)
    %loaded_int3 = load i32, ptr %b2, align 4
    call void (ptr, ...) @printf(ptr @1, i32 %loaded_int3)
    %loaded_int4 = load i32, ptr %result, align 4
    call void (ptr, ...) @printf(ptr @2, i32 %loaded_int4)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %calltmp = call i32 @max(i32 10, i32 20)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

declare void @printf(ptr, ...)

```

Листинг 3.22 – Сгенерированный LLVM IR код для тестовой программы №8

```

; ModuleID = 'generated'

```

```

source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [10 x i8] c"a + b > 0\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@2 = private unnamed_addr constant [11 x i8] c"a + b <= 0\00", align 1
@3 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@4 = private unnamed_addr constant [10 x i8] c"a + b > 0\00", align 1
@5 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@6 = private unnamed_addr constant [11 x i8] c"a + b <= 0\00", align 1
@7 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %calltmp = call i32 @someFunc(i32 1982, i32 3)
    %calltmp1 = call i32 @someFunc(i32 0, i32 0)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @someFunc(i32 %a, i32 %b) {
entry:
    %a1 = alloca i32, align 4
    store i32 %a, ptr %a1, align 4
    %b2 = alloca i32, align 4
    store i32 %b, ptr %b2, align 4
    %retval = alloca i32, align 4
    %loaded_lhs = load i32, ptr %a1, align 4
    %loaded_rhs = load i32, ptr %b2, align 4
    %addtmp = add i32 %loaded_lhs, %loaded_rhs
    %gttmp = icmp sgt i32 %addtmp, 0
    %ifcond = icmp ne i1 %gttmp, false
    br i1 %ifcond, label %if.then, label %if.else

if.then:                                     ; preds = %entry
    call void (ptr, ...) @printf(ptr @1, ptr @0)
    br label %if.end

if.end:                                     ; preds = %if.else, %if.then
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret

if.else:                                     ; preds = %entry
    call void (ptr, ...) @printf(ptr @3, ptr @2)
    br label %if.end

```

```

entry3:                                     ; No predecessors!
    %a4 = alloca i32, align 4
    store i32 %a, ptr %a4, align 4
    %b5 = alloca i32, align 4
    store i32 %b, ptr %b5, align 4
    %retval6 = alloca i32, align 4
    %loaded_lhs7 = load i32, ptr %a4, align 4
    %loaded_rhs8 = load i32, ptr %b5, align 4
    %addtmp9 = add i32 %loaded_lhs7, %loaded_rhs8
    %gttmp10 = icmp sgt i32 %addtmp9, 0
    %ifcond11 = icmp ne i1 %gttmp10, false
    br i1 %ifcond11, label %if.then12, label %if.else14

if.then12:                                 ; preds = %entry3
    call void @printf(ptr @5, ptr @4)
    br label %if.end13

if.end13:                                  ; preds = %if.else14, %if.then12
    %ret15 = load i32, ptr %retval6, align 4
    ret i32 %ret15

if.else14:                                 ; preds = %entry3
    call void @printf(ptr @7, ptr @6)
    br label %if.end13
}

declare void @printf(ptr, ...)

```

Листинг 3.23 – Сгенерированный LLVM IR код для тестовой программы №9

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %x = alloca i32, align 4
    store i32 2, ptr %x, align 4
    %loaded_from_var = load i32, ptr %x, align 4
    store i32 %loaded_from_var, ptr %x, align 4

```

```

switch_cond = load i32, ptr %x, align 4
switch i32 %switch_cond, label %switch.end [
    i32 1, label %case
    i32 2, label %case1
    i32 3, label %case2
]

switch.end:                                     ; preds = %case2, %case1, %case, %entry
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret

case:                                           ; preds = %entry
    call void (ptr, ...) @printf(ptr @0, i32 100)
    br label %switch.end

case1:                                         ; preds = %entry
    call void (ptr, ...) @printf(ptr @1, i32 200)
    br label %switch.end

case2:                                         ; preds = %entry
    call void (ptr, ...) @printf(ptr @2, i32 300)
    br label %switch.end
}

declare void @printf(ptr, ...)

```

Листинг 3.24 – Сгенерированный LLVM IR код для тестовой программы №10

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@3 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %x = alloca i32, align 4
    store i32 5, ptr %x, align 4
    %loaded_from_var = load i32, ptr %x, align 4
    store i32 %loaded_from_var, ptr %x, align 4
    %calltmp = call i32 @goT0func(i32 0)
    %x1 = alloca i32, align 4

```

```

    store i32 3, ptr %x1, align 4
    %loaded_from_var2 = load i32, ptr %x1, align 4
    store i32 %loaded_from_var2, ptr %x1, align 4
    %calltmp3 = call i32 @goT0func(i32 4)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
}

define i32 @goT0func(i32 %y) {
entry:
    %y1 = alloca i32, align 4
    store i32 %y, ptr %y1, align 4
    %retval = alloca i32, align 4
    %loaded_lhs = load i32, ptr %y1, align 4
    %gttmp = icmp sgt i32 %loaded_lhs, 0
    %ifcond = icmp ne i1 %gttmp, false
    br i1 %ifcond, label %if.then, label %if.end

if.then:                                     ; preds = %entry
    br label %label1

if.end:                                     ; preds = %after_goto, %entry
    call void (ptr, ...) @printf(ptr @0, i32 0)
    br label %label1

label1:                                     ; preds = %if.end9, %if.then8, %if.end,
    %if.then
    call void (ptr, ...) @printf(ptr @1, i32 42)
    %ret = load i32, ptr %retval, align 4
    ret i32 %ret
    call void (ptr, ...) @printf(ptr @3, i32 42)
    %ret11 = load i32, ptr %retval4, align 4
    ret i32 %ret11

after_goto:                                ; No predecessors!
    br label %if.end

entry2:                                    ; No predecessors!
    %y3 = alloca i32, align 4
    store i32 %y, ptr %y3, align 4
    %retval4 = alloca i32, align 4
    %loaded_lhs5 = load i32, ptr %y3, align 4
    %gttmp6 = icmp sgt i32 %loaded_lhs5, 0
    %ifcond7 = icmp ne i1 %gttmp6, false
    br i1 %ifcond7, label %if.then8, label %if.end9

if.then8:                                  ; preds = %entry2
    br label %label1

```

```

if.end9:                                     ; preds = %after_goto10, %entry2
    call void @printf(ptr @2, i32 0)
    br label %label1

after_goto10:                               ; No predecessors!
    br label %if.end9
}

declare void @printf(ptr, ...)

```

### Листинг 3.25 – Сгенерированный LLVM IR код для тестовой программы №11

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %max = alloca i32, align 4
    store i32 2, ptr %max, align 4
    %loaded_from_var = load i32, ptr %max, align 4
    store i32 %loaded_from_var, ptr %max, align 4
    %x = alloca i32, align 4
    store i32 10, ptr %x, align 4
    %loaded_from_var1 = load i32, ptr %x, align 4
    store i32 %loaded_from_var1, ptr %x, align 4
    %y = alloca i32, align 4
    store i32 20, ptr %y, align 4
    %loaded_from_var2 = load i32, ptr %y, align 4
    store i32 %loaded_from_var2, ptr %y, align 4
    %loaded_cond = load i32, ptr %y, align 4
    %cond_bool = icmp ne i32 %loaded_cond, 0
    br i1 %cond_bool, label %ternary.true, label %ternary.false

ternary.true:                               ; preds = %entry
    br label %ternary.merge

ternary.false:                             ; preds = %entry
    br label %ternary.merge

ternary.merge:                             ; preds = %ternary.false, %ternary.true
    %ternarytmp = phi ptr [ %x, %ternary.true ], [ %y, %ternary.false ]

```



```

%loaded_lhs = load i32, ptr %x, align 4
%loaded_rhs = load i32, ptr %ternarytmp, align 4
%gttmp = icmp sgt i32 %loaded_lhs, %loaded_rhs
%select_result = select i1 %gttmp, ptr %x, ptr %y
%selected_loaded = load i32, ptr %select_result, align 4
store i32 %selected_loaded, ptr %max, align 4
%loaded_int = load i32, ptr %max, align 4
call void (ptr, ...) @printf(ptr @0, i32 %loaded_int)
%ret = load i32, ptr %retval, align 4
ret i32 %ret
}

```

```
declare void @printf(ptr, ...)
```

Листинг 3.26 – Сгенерированный LLVM IR код для тестовой программы №12

```

; ModuleID = 'generated'
source_filename = "generated"
target datalayout =
    "e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-n32:64-S128-Fn32"
target triple = "arm64-apple-darwin23.4.0"

@0 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
@1 = private unnamed_addr constant [2 x i8] c" \00", align 1
@2 = private unnamed_addr constant [4 x i8] c"%s\0A\00", align 1
@3 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

define i32 @main() {
entry:
    %retval = alloca i32, align 4
    %arr = alloca [5 x i32], align 4
    %arr_decay = getelementptr [5 x i32], ptr %arr, i32 0, i32 0
    %arr_elem = getelementptr i32, ptr %arr_decay, i32 0
    store i32 1, ptr %arr_elem, align 4
    %arr_elem1 = getelementptr i32, ptr %arr_decay, i32 1
    store i32 2, ptr %arr_elem1, align 4
    %arr_elem2 = getelementptr i32, ptr %arr_decay, i32 2
    store i32 3, ptr %arr_elem2, align 4
    %arr_elem3 = getelementptr i32, ptr %arr_decay, i32 3
    store i32 4, ptr %arr_elem3, align 4
    %arr_elem4 = getelementptr i32, ptr %arr_decay, i32 4
    store i32 5, ptr %arr_elem4, align 4
    %i = alloca i32, align 4
    store i32 0, ptr %i, align 4
    %loaded_from_var = load i32, ptr %i, align 4
    store i32 %loaded_from_var, ptr %i, align 4
    %j = alloca i32, align 4
    store i32 4, ptr %j, align 4

```

```

%loaded_from_var5 = load i32, ptr %j, align 4
store i32 %loaded_from_var5, ptr %j, align 4
%tmpCounter = alloca i32, align 4
store i32 0, ptr %tmpCounter, align 4
%loaded_from_var6 = load i32, ptr %tmpCounter, align 4
store i32 %loaded_from_var6, ptr %tmpCounter, align 4
br label %while.cond

while.cond:                                     ; preds = %while.body, %entry
%loaded_lhs = load i32, ptr %tmpCounter, align 4
%ltmp = icmp slt i32 %loaded_lhs, 5
%whilecond = icmp ne i1 %ltmp, false
br i1 %whilecond, label %while.body, label %while.end

while.body:                                     ; preds = %while.cond
%loaded_index = load i32, ptr %tmpCounter, align 4
%arr_elem7 = getelementptr i32, ptr %arr_decay, i32 %loaded_index
%arr_val = load i32, ptr %arr_elem7, align 4
call void (ptr, ...) @printf(ptr @0, i32 %arr_val)
%loadtmp = load i32, ptr %tmpCounter, align 4
%inctmp = add i32 %loadtmp, 1
store i32 %inctmp, ptr %tmpCounter, align 4
br label %while.cond

while.end:                                     ; preds = %while.cond
br label %while.cond8

while.cond8:                                   ; preds = %while.body9, %while.end
%loaded_lhs11 = load i32, ptr %i, align 4
%loaded_rhs = load i32, ptr %j, align 4
%ltmp12 = icmp slt i32 %loaded_lhs11, %loaded_rhs
%whilecond13 = icmp ne i1 %ltmp12, false
br i1 %whilecond13, label %while.body9, label %while.end10

while.body9:                                   ; preds = %while.cond8
%loaded_index14 = load i32, ptr %i, align 4
%arr_elem15 = getelementptr i32, ptr %arr_decay, i32 %loaded_index14
%arr_val16 = load i32, ptr %arr_elem15, align 4
%tmp1 = alloca i32, align 4
store i32 %arr_val16, ptr %tmp1, align 4
%loaded_index17 = load i32, ptr %j, align 4
%arr_elem18 = getelementptr i32, ptr %arr_decay, i32 %loaded_index17
%arr_val19 = load i32, ptr %arr_elem18, align 4
%tmp2 = alloca i32, align 4
store i32 %arr_val19, ptr %tmp2, align 4
%loaded_index20 = load i32, ptr %i, align 4
%arr_elem21 = getelementptr i32, ptr %arr_decay, i32 %loaded_index20
%arr_loaded_rhs = load i32, ptr %tmp2, align 4

```

```

store i32 %arr_loaded_rhs, ptr %arr_elem21, align 4
%loaded_index22 = load i32, ptr %j, align 4
%arr_elem23 = getelementptr i32, ptr %arr_decay, i32 %loaded_index22
%arr_loaded_rhs24 = load i32, ptr %tmp1, align 4
store i32 %arr_loaded_rhs24, ptr %arr_elem23, align 4
%loadtmp25 = load i32, ptr %i, align 4
%inctmp26 = add i32 %loadtmp25, 1
store i32 %inctmp26, ptr %i, align 4
%loadtmp27 = load i32, ptr %j, align 4
%dectmp = sub i32 %loadtmp27, 1
store i32 %dectmp, ptr %j, align 4
br label %while.cond8

while.end10:                                     ; preds = %while.cond8
call void (ptr, ...) @printf(ptr @2, ptr @1)
%tmpCounter28 = alloca i32, align 4
store i32 0, ptr %tmpCounter28, align 4
%loaded_from_var29 = load i32, ptr %tmpCounter28, align 4
store i32 %loaded_from_var29, ptr %tmpCounter28, align 4
br label %while.cond30

while.cond30:                                     ; preds = %while.body31, %while.end10
%loaded_lhs33 = load i32, ptr %tmpCounter28, align 4
%ltmp34 = icmp slt i32 %loaded_lhs33, 5
%whilecond35 = icmp ne i1 %ltmp34, false
br i1 %whilecond35, label %while.body31, label %while.end32

while.body31:                                     ; preds = %while.cond30
%loaded_index36 = load i32, ptr %tmpCounter28, align 4
%arr_elem37 = getelementptr i32, ptr %arr_decay, i32 %loaded_index36
%arr_val38 = load i32, ptr %arr_elem37, align 4
call void (ptr, ...) @printf(ptr @3, i32 %arr_val38)
%loadtmp39 = load i32, ptr %tmpCounter28, align 4
%inctmp40 = add i32 %loadtmp39, 1
store i32 %inctmp40, ptr %tmpCounter28, align 4
br label %while.cond30

while.end32:                                     ; preds = %while.cond30
%ret = load i32, ptr %retval, align 4
ret i32 %ret
}

declare void @printf(ptr, ...)

```